

Théorie des langages

Olivier Ridoux — 2023

Motivations

Les systèmes informatiques permettent de réaliser de façon automatique un éventail toujours plus varié de tâches. Celles-ci commencent souvent par lire des données selon un format d'entrée, pour ensuite en calculer d'autres selon un autre format. La **théorie des langages (TL)** permet d'étudier formellement dans quelles conditions ces tâches peuvent être automatisées. Comment **lire** une donnée et décider si elle est au bon format ? Comment **produire** une donnée qui respecte un format attendu ? Et à la base de tout, comment **caractériser** un format ?

Cette motivation d'ingénierie informatique n'est pas la motivation historique du développement de la théorie des langages dans les années 1930-50. Deux sources indépendantes en sont l'origine :

1. Formaliser des questions **linguistiques**. Comment caractériser une langue naturelle ? Comment reconnaître qu'une phrase est correctement formée ? Les premiers modèles formels qui ont été proposés dans ce but se sont révélés décevants pour cet usage, mais bien adaptés aux applications informatiques qui sont apparues plus tard. Ce sont ceux-ci qui sont présentés ici.
2. Formaliser des questions de **télécommunication**. Comment reconnaître qu'un message n'a pas été altéré par le bruit de transmission ? Comment décrire l'organisation des messages et de leur signalisation, numérotation, etc. ? Comment retrouver les éléments de signalisation dans un message ?

La TL répond à ces questions par des résultats formels très puissants qui ont permis de développer des outils qui sont dans tous les systèmes informatiques : compilateurs, gestionnaires de documents structurés, etc. La TL offre donc un magnifique exemple de **théorie qui marche** propre à une **bonne ingénierie**.

Ce module est accompagné de travaux pratiques utilisant des outils modernes comme **Xtext**.

Lettres, mots et langages

Comme toutes les mathématiques, la TL utilise des termes de la langue commune pour désigner des concepts spécifiques. Il faut utiliser ces termes formels avec exactitude sans les confondre avec les termes homonymes du sens commun.

Lettre (resp. **symbole**) : Il s'agit de signes distincts deux à deux. Un **alphabet** (resp. **vocabulaire**), souvent noté Σ (resp. V), est un ensemble non vide de lettres, toujours fini dans la théorie décrite ici. Les lettres génériques sont souvent notées $a, b, \dots$. On appelle **lettre-X** (resp. **symbole-X**) une lettre formelle qui est un X au sens commun.

Le concept formel de lettre évoque celui de lettre d'un alphabet au sens commun, mais on peut l'interpréter beaucoup plus largement, ex. par des fréquences sonores. L'important est que l'ensemble des signes soit fini et que les signes soient aisément distinguables. Ex. les **lettres-fréquences** pourront être 262 Hz, 277 Hz, 294 Hz, ..., 523 Hz, mais pas le continuum des fréquences entre 262 et 523 Hz.

Mot : Un mot est une séquence de lettres, qui peut être vide, et qui dans la théorie décrite ici est toujours finie. Dans un mot non vide il y a une première lettre, une seconde, etc., et une dernière lettre. On note usuellement un mot générique par un w (pour **word**), ou les lettres voisines, un mot particulier non vide par la suite de ses lettres, ex. a ou **abab**, et le mot vide par ϵ . On dira **mot-X** pour désigner des mots formels qui sont des X au sens commun. Ex. une séquence de **lettres-fréquences** pourrait former un **mot-mélodie**, et une séquence de **lettres-mots** pourrait former un **mot-phrase** (où on voit que le « mot » du sens commun n'est pas forcément un mot formel).

La théorie des langages confère aux mots une structure algébrique en les dotant d'une opération appelée **concaténation** et notée $\bullet$. Elle consiste à mettre bout à bout deux mots pour en former un autre. Ex. **ab** $\bullet$ **cd** = **abcd**. La concaténation est associative, ce qui permet d'écrire **u** $\bullet$ **v** $\bullet$ **w** sans parenthèses, et même **uvw**. Elle a un élément neutre à gauche et à droite, qui est ϵ . Elle n'est **pas commutative**.

L'ensemble de tous les mots qu'il est possible de former avec les lettres d'un vocabulaire V (resp. Σ) est noté V^* (resp. Σ^*). Cet ensemble est infini même si ses éléments sont des mots finis. Il est **dénombrable**.

Langage : Un langage est une partie de V^* . Les langages sont habituellement notés $\mathcal{L}$. Étant donné un vocabulaire V , on peut former une infinité de langages, dont $\emptyset$ et V^* . L'ensemble constitué de tous les langages **n'est pas dénombrable**.

La TL confère une structure algébrique aux langages issus d'un même vocabulaire en les dotant d'opérations. L'une d'entre elles, appelée **produit** et notée $\bullet$, permet de former un nouveau langage en concaténant les mots de deux langages :

$$\mathcal{L}_1 \bullet \mathcal{L}_2 = \{ w_1 \bullet w_2 \mid w_1 \in \mathcal{L}_1 \wedge w_2 \in \mathcal{L}_2 \}.$$

Le produit est associatif et a $\{\epsilon\}$ pour élément neutre à gauche et à droite. Il n'est **pas commutatif**. Le langage vide est un élément absorbant, $\mathcal{L} \bullet \emptyset = \emptyset \bullet \mathcal{L} = \emptyset$! Et ϵ n'est pas un langage. Confondre ϵ , $\{\epsilon\}$ et $\emptyset$ constitue donc une grave erreur, comme le serait confondre le chiffre '0' et les nombres 0 et 1 pour la **multiplication** usuelle.

Une autre opération, appelée **fermeture** et notée $*$, permet de former un nouveau langage par itération de la concaténation :

$$\mathcal{L}^* = \{\epsilon\} \cup \mathcal{L} \cup \mathcal{L} \bullet \mathcal{L} \cup \mathcal{L} \bullet \mathcal{L} \bullet \mathcal{L} \cup \dots$$

$\mathcal{L}^*$ est toujours infini, sauf si $\mathcal{L}$ est vide. Cet opérateur est aussi appelé « **étoile de Kleene** » du nom du mathématicien Stephen Kleene (1909–1994) à qui on doit de nombreux résultats de la théorie des langages.

Les langages intéressants sont ceux qui ne sont ni $\emptyset$ ni V^* , et qui sont gros voire infinis. On ne peut donc pas les représenter en extension, et c'est une contribution importante de la théorie des langages que de proposer des moyens finis et compacts pour représenter des langages gros voire infinis. Ex. un langage formel comme **Java** est l'ensemble des **mots-programmes** qui respectent la spécification de ce langage. Cet ensemble est gros, même infini, et correspond bien à la notion informelle de langage intéressant. Mais l'enjeu de la TL n'est pas simplement de caractériser Java, mais plutôt de caractériser des **moyens** de caractériser des langages, de **classifier** ces moyens, et d'en déterminer le **pouvoir d'expression**. Ex. les opérations ci-dessus suffisent-elles à caractériser le langage Java ?

Comment caractériser formellement un langage ?

La TL propose deux styles de caractérisation d'un langage :

1. Former des langages plus gros en composant des langages plus petits. C'est un style **extensionnel**, adapté aux humains.
2. Donner une procédure de décision qu'un mot appartient à un langage. C'est un style **intensionnel**, adapté aux machines.

L'idéal serait bien sûr d'avoir une **passerelle** entre les deux, de façon à ce qu'un humain puisse caractériser commodément, de son point de vue, un langage dans un style extensionnel, puis traduire cette caractérisation en style intensionnel plus commode pour une machine. Beaucoup d'exemples de passerelle existent, et c'est tout l'intérêt de la TL que de pouvoir le démontrer formellement (ex. page 5).

Expressions régulières (re, pour *regular expression*) : Elles sont un exemple du style **extensionnel** pour caractériser les langages. La théorie décrit comment former des re, et quels langages celles-ci dénotent. On note une re générique r , et **RE** l'ensemble des re qu'il est possible de former. RE est **dénombrable**.

Soit $V = \{a, b, \dots\}$, les re de base sont $\underline{\epsilon}$, $\underline{a}$, $\underline{b}$, ... Elles décrivent les petits langages qui en composent de plus gros. Le soulignement sert ici à distinguer l'utilisation des lettres du **langage décrit** dans les **re qui le décrivent**. Soit r , r_1 et r_2 des re, $r_1 | r_2$, $r_1 r_2$ et r^* sont aussi des re. On pourra utiliser des parenthèses pour marquer la portée des $|$ et des $*$. Les re dénotent des langages de la façon suivante :

- $SEM(\underline{\epsilon}) = \{\epsilon\}$ $SEM(\underline{a}) = \{a\}$ $SEM(\underline{b}) = \{b\}$...
- $SEM(r_1 | r_2) = SEM(r_1) \cup SEM(r_2)$ union de deux langages
- $SEM(r_1 r_2) = SEM(r_1) \cdot SEM(r_2)$ produit de deux langages
- $SEM(r^*) = SEM(r)^*$ fermeture d'un langage

RE constitue donc un langage qui décrit des langages ; c'est un **métalangage**.

Ex. la re **###4/4(CDFCBCACCBFAFFCCB)*(EEEEFBBBBBBBBBBBAAAAA)*CDECBCA** décrit les premières notes du *YELLOW SUBMARINE* des Beatles (C=Do, D=Ré, etc.) où V est $\{\#, 4, /, A, B, C, D, E, F, G\}$. Ici, la re engendre des **mots-partitions**.

On qualifie de **rationnels** les langages qu'il est possible de caractériser par une re. L'ensemble des langages n'étant pas dénombrable mais RE l'étant, les langages ne sont pas tous rationnels ! Mais alors, lesquels le sont ? Lesquels ne le sont pas ?

Automates finis (fsa, pour *finite state automata*) : Ils sont un exemple du style **intensionnel** de caractérisation des langages. La théorie décrit comment former des fsa, et comment ceux-ci **acceptent** des mots. On note un fsa générique M (pour *Machine*). Chaque fsa est fini. Les fsa constituent un ensemble **dénombrable** : **FSA**.

Un fsa est défini par un vocabulaire V , un ensemble fini d'états, noté Q , contenant un état initial $q_0 \in Q$, et des états finaux $F \subseteq Q$. Il est aussi défini par une relation de transition δ constituée d'un ensemble de triplets $(q, \underline{a}, q')$, souvent notés $q \xrightarrow{a} q'$, où $a \in V$ et $q, q' \in Q$. Les fsa **acceptent** des mots de la façon suivante :

- $ACCEPT(\epsilon, q)$ est vrai ssi $q \in F$
les états finaux acceptent le mot vide, ils sont les seuls à le faire
- $ACCEPT(a \cdot m, q)$ est vrai ssi $\exists q' \text{ tq } q \xrightarrow{a} q' \cdot ACCEPT(m, q')$
un état quelconque accepte un mot si il permet une transition qui accepte sa première lettre et qui conduit à un état qui accepte le reste
- un fsa **accepte** un mot m ssi $ACCEPT(m, q_0)$.

Un fsa **reconnaît** un langage si il en accepte tous les mots, et seulement eux.

Théorème de Kleene : Les langages rationnels sont les langages reconnus par les fsa, et vice-versa. Cela rend possible la passerelle attendue entre les re, adaptées aux humains, et les fsa, adaptés aux machines. Cette passerelle est réalisée dans des outils comme **Perl**, le **grep** du shell UNIX, le **REGEXP** de SQL, **LEX** et **Xtext**.

On a vu que les langages ne pouvaient pas tous être rationnels ; comment distinguer ceux qui ne le sont pas ? Le lemme suivant donne un critère.

Lemme de l'étoile (simplifié) : Si un langage est rationnel alors chacun de ses mots plus longs qu'une limite qui dépend du langage peut être découpé en 3 tronçons $u \cdot v \cdot w$ tels que u n'est pas trop long et $\{u\} \cdot \{v\}^* \cdot \{w\}$ est inclus dans le langage. On démontre qu'un langage n'est pas rationnel en prenant la **contraposée** du critère. En fait, de nombreux langages ne sont pas rationnels :

- **Langage des palindromes** : le langage des mots qui se lisent dans les deux sens (ex. 252, kayak et ressasser).
- **Langage des parenthèses** : le langage des mots balisés par des parenthèses bien imbriquées, ex. $(...)(((((...)))(...)))(...))$, quel que soit le degré d'imbrication.
- **RE** : à cause des parenthèses qui marquent la portée des $|$ et des $*$. Le métalangage qui décrit les langages rationnels n'est donc pas rationnel. Et Java non plus, à cause de ses parenthèses et de ses accolades.

Avec le Lemme de l'étoile se clôt cette présentation des langages rationnels. Il faut surtout en retenir l'architecture générale : l'échafaudage **lettre-mot-langage**, les moyens de **caractériser des langages**, par **expression** (extensionnelle) et par **automate** (intensionnel), l'**équivalence** formelle entre ces moyens, l'identification de leur **puissance d'expression**, et la formalisation de cette puissance d'expression dans un **critère d'appartenance**.

La TL ne s'arrête pas aux langages rationnels. Que certains langages ne soient pas rationnels fait se demander si on peut construire le même genre d'édifice pour eux, ou une partie d'entre eux. Et la réponse est oui. Par exemple, on montre que le langage des parenthèses est l'archétype d'une famille de langages qui sont bien caractérisés par une autre forme d'expression, les **grammaires algébriques**, et par des **automates à pile**, que les deux ont exactement la même puissance d'expression, qui est bien testée par une variante du Lemme de l'étoile, le tout constituant la théorie des **langages algébriques** (on dit aussi **langage sans contexte**). On montre aussi que les langages ne sont pas tous algébriques, et on recommence en définissant des classes de langages toujours plus larges qu'on ordonne dans la **Hiérarchie de Chomsky**, du nom de Noam Chomsky (1928-...), autre grand nom de la TL. Dans cette hiérarchie, les classes de langages sont désignées par un numéro ; ex. les langages rationnels sont les langages de **type 3** (voir page 7).

Analyse syntaxique

Jusque là l'enjeu était de caractériser l'appartenance d'un mot à un langage. Mais ce n'est pas suffisant car les applications qui lisent des données formatées veulent en plus utiliser le format comme un guide pour leur calcul.

Depuis Chomsky, on privilégie des formes de caractérisation des langages qu'on appelle **grammaire**. On y trouve les re, avec les lettres d'un vocabulaire, plus un second vocabulaire qui fournit les noms des composants structurels d'un document. On appelle le premier le vocabulaire **terminal**, noté **T**, et le second le vocabulaire **non-terminal**, noté **NT**. **T** correspond au lexique d'une langue naturelle, ex. **je**, **qui**, **le**, **pomme**, **aime**, **Alice**, et **NT** à ses structures grammaticales, ex. **pronom**, **article**, **nom-commun**, **verbe**, **nom-propre**. Enfin, des **règles grammaticales** définissent comment chaque composant structurel est formé :

phrase $\rightarrow$ sujet verbe complément
sujet $\rightarrow$ pronom-sujet | nom
nom $\rightarrow$ article nom-commun | nom-propre
pronom-sujet $\rightarrow$ je | tu | il | ...
verbe $\rightarrow$ aime | apprend | ...
complément $\rightarrow$ nom (ϵ | qui verbe complément)

Noter comment **complément** est défini en fonction de lui-même. Cette forme de grammaire est appelée **grammaire algébrique**, noté **GA**. D'autres formes existent.

Une grammaire **engendre** un langage de la façon suivante :

- $SEM(nt) = \bigcup_{nt \rightarrow r} SEM(r)$ où $nt \in NT$
- $SEM(r) = \dots$ comme pour les expressions régulières
- La grammaire **engendre** w ssi $w \in SEM(\text{phrase})$

Cette grammaire engendre donc des mots-phrases comme **Alice aime le violon**, mais aussi **Bob aime Alice qui apprend le violon**. L'enjeu de l'analyse syntaxique est de reconnaître que dans la première phrase **Alice** est le **sujet** et **le violon** est le **complément**, mais que dans la seconde **Alice** est une partie du **complément**.

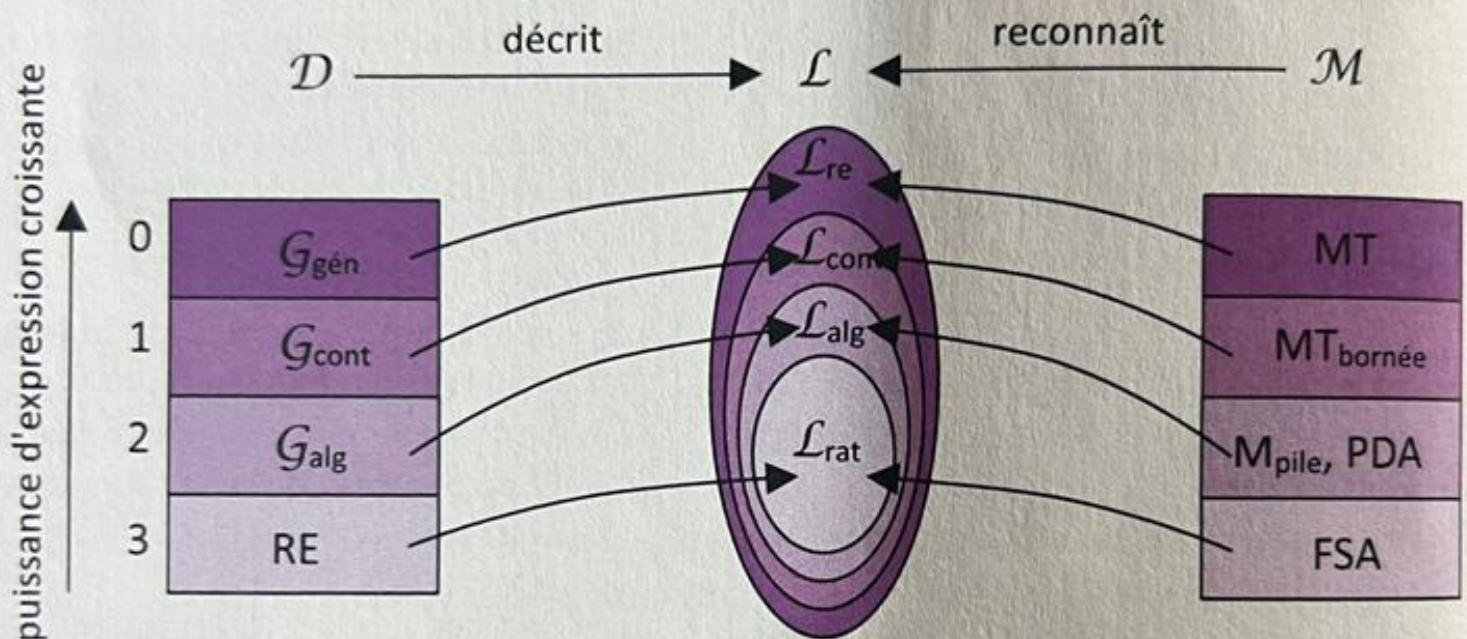
L'analyse syntaxique présente deux difficultés, l'une liée aux grammaires, l'autre au processus d'analyse :

1. **Ambiguïté** : une grammaire peut donner plusieurs décompositions structurelles à la même phrase, comme dans **Alice a vu Bob avec sa longue-vue** (de quoi avec sa longue-vue est-il le **complément** ?). À éviter absolument !
2. **Incomplétude** : il existe des algorithmes qui réalisent bien l'analyse syntaxique dans tous les cas, mais on leur préfère souvent des variantes plus simples qui ne marchent que pour certaines familles de grammaires. Lire la documentation !

2x2 formalismes linguistiques

	sans imbrication	avec imbrication
ensemble (extensionnel)	expressions régulières : ex. format CSV	grammaires algébriques : ex. langage WHILE, format CSV
reconnaisseur (intensionnel)	automates finis : ex. format CSV	automates à pile : ex. langage WHILE, format CSV

Hiérarchie de Chomsky



Type	Grammaire Chomsky, 1959	Automate	Langage
0	$\Sigma_T \mid \Sigma_N)^+ \rightarrow (\Sigma_T \mid \Sigma_N)^*$	MT Turing, 1936	rékursivement énumérable Post, 1947
1	$\alpha A \beta \rightarrow \alpha \gamma \beta$	MT bornée ..., Kuroda, 1964	avec contexte
2	$\Sigma_N \rightarrow (\Sigma_T \mid \Sigma_N)^*$ algébrique	à pile Chomsky, 1962	sans contexte 1947-64
3	$\Sigma_N \rightarrow \Sigma_T \Sigma_N \mid \epsilon$ régulière (RE !)	fini Kleene, 1951-56	rationnel Kleene, 1951-56

